



INTEGRAR

 SESIÓN 13 · UNIDAD III · CIERRE

CERRAR EL CÍRCULO DE LA dAPP

Leer la historia sin recorrer la cadena, entrar sin contraseña con una firma, y entregar un proyecto que otro pueda correr.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Apertura de 10 minutos y una pausa de 10 entre A y B. Hoy la teoría es corta y práctica: todo lo que se explica en A se usa en B o se revisa en C.

A**INFRAESTRUCTURA DEL ECOSISTEMA**

Indexación, ENS, EIP-712 y SIWE, abstracción de cuentas, hosting y repositorio.

~65 min**B****LAB 13 + INTEGRACIÓN ASISTIDA**

Autenticación por firma EIP-712; después, cada equipo integra su dApp.

~70 min**C****ENTREGA DEL AVANCE 1**

Lista de chequeo, rúbrica y cierre de la revisión técnica por equipos.

~25 min



AL SALIR DE AQUÍ

Cerrar el ciclo de la dApp e incorporar la infraestructura que la hace usable: indexación, nombres, firmas y cuentas.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Por qué no se puede sacar el historial de un contrato leyendo su estado, y qué se usa en su lugar?
- 02** ¿Qué firma exactamente una billetera en EIP-712, y qué impide que esa firma se reutilice?
- 03** ¿Cómo se inicia sesión con una firma, y qué hace la abstracción de cuentas por el usuario nuevo?
- 04** ¿Qué tiene que tener un repositorio para que otro lo corra sin preguntarles nada?

RA4 · despliegue verificado y reproducible · RA6 · integración de la interfaz con contratos, firmas y eventos.



SIGLAS Y TÉRMINOS DE HOY

LOG / EVENTO*registro de ejecución*

Dato que un contrato emite con emit. Queda en el recibo, no en el estado: el contrato no puede leerlo después.

SUBGRAFO*subgraph*

La receta de indexación para The Graph: qué contrato, qué eventos y cómo guardarlos. Se consulta con GraphQL.

EIP-712*typed structured data*

Estándar para firmar datos con tipos y un dominio. La billetera muestra campos legibles, no bytes.

ERC-4337*account abstraction*

Cuentas que son contratos, operadas con UserOperations, bundlers y un contrato EntryPoint.

INDEXADOR*indexer*

Servicio que lee los eventos una vez, los guarda en una base de datos y responde consultas rápidas.

ENS*Ethereum Name Service*

Registro en cadena que traduce nombres como vitalik.eth a direcciones, y al revés.

SIWE · ERC-4361*Sign-In with Ethereum*

Iniciar sesión firmando un mensaje de texto estándar con nonce y dominio. Sin contraseña.

EIP-7702*set code for EOAs*

Desde mayo de 2025, una cuenta normal puede delegar su ejecución a un contrato sin cambiar de dirección.

A



APROXIMADAMENTE 65 MINUTOS

INFRAESTRUCTURA DEL ECOSISTEMA

Un contrato y una pantalla bastan para una demo. Para que la use gente real faltan piezas: leer la historia, nombres legibles, firmas y cuentas amables.



EL ESTADO SOLO GUARDA EL PRESENTE

La interfaz de la Sesión 12 leía `balanceOf`: una llamada, instantánea, gratis. Pero el producto pide otra cosa: «muéstrame todas las asistencias de Ana» o «las diez últimas transferencias». Y eso el contrato no lo sabe.

LO QUE GUARDA EL ESTADO

- El VALOR ACTUAL de cada variable: saldo de hoy, dueño de hoy.
- Cada escritura pisa la anterior: el valor viejo ya no está en el storage.
- Leerlo es barato: una llamada por dato.

DÓNDE ESTÁ LA HISTORIA

- En los LOGS de los recibos: cada emit deja un registro con el bloque y la transacción.
- Los logs no son accesibles desde el contrato: solo desde fuera.
- Filtrarlos exige pedir rangos de bloques al nodo.

ERROR TÍPICO

«Guardo un arreglo con todo el historial en el contrato y lo leo desde la interfaz». Funciona con diez registros y se vuelve caro e ilegible con diez mil. La historia va en eventos; el estado guarda solo lo que el contrato necesita para decidir.



RECORRER UN AÑO DE BLOQUES, A MANO

Supongamos que la interfaz quiere el historial completo de un contrato que lleva un año desplegado. En Ethereum sale un bloque cada 12 segundos.

2 628 000

bloques en un año: $365 \times 24 \times 3\,600 \div 12$. Todos hay que revisarlos buscando los logs del contrato.

263

consultas `eth_getLogs` si el proveedor deja pedir 10 000 bloques por vez. En cada visita de cada usuario.

0

capacidad de ordenar, agregar o paginar en el nodo: eso lo tiene que hacer el navegador con todo en memoria.

Cada proveedor de RPC limita el rango de bloques o el número de resultados por consulta, y cuenta las llamadas contra su plan gratuito. Lo que en local es instantáneo, en Sepolia o en la red principal se vuelve lento, caro o simplemente rechazado.

La salida es indexar: leer los eventos UNA vez, guardarlos en una base de datos, y que la interfaz consulte esa base.

LEER EVENTOS CON QUERYFILTER

DEL CONTRATO A LA CONSULTA

```
// el evento del laboratorio de hoy: tres campos indexed = tres «topics» filtrables
event AsistenciaRegistrada(address indexed estudiante, uint256 indexed sesion,
                           address indexed enviadaPor, uint256 nonce);

// ethers v6, desde la interfaz o un script
const filtro = contrato.filters.AsistenciaRegistrada(ana); // solo las de Ana
const eventos = await contrato.queryFilter(filtro, bloqueDespliegue, "latest");
for (const e of eventos) console.log(e.blockNumber, e.args.sesion, e.args.nonce);
```

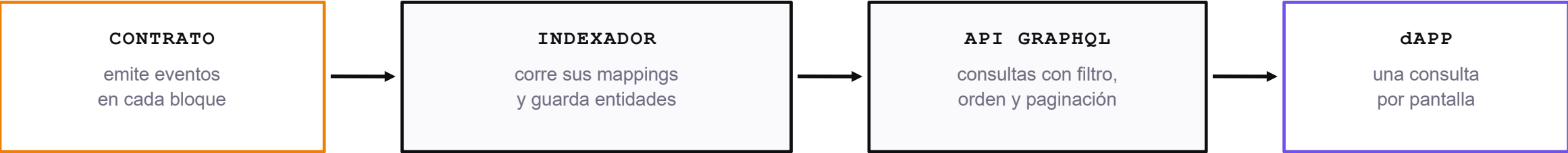
CUÁNDO BASTA

- Pocos eventos y un solo contrato: un proyecto de curso.
- Empezar la búsqueda en el bloque del despliegue, no en 0.
- Filtrar por campos indexed: el nodo hace el trabajo.

CUÁNDO YA NO

- Miles de eventos o varios contratos a la vez.
- Hace falta ordenar, sumar o paginar.
- Cada usuario repetiría las mismas consultas al RPC.

THE GRAPH: UN SUBGRAFO EN TRES ARCHIVOS



ARCHIVO	QUÉ DICE
subgraph.yaml	Qué contrato, en qué red, desde qué bloque y qué eventos escuchar (el manifiesto).
schema.graphql	Qué entidades se guardan: por ejemplo Asistencia { estudiante, sesion, bloque }.
mapping.ts	Qué hacer con cada evento: crear o actualizar entidades. Se escribe en AssemblyScript.

 ESTADO ACTUAL, VERIFICADO

El «hosted service» gratuito de The Graph se apagó el 12 de junio de 2024. Hoy se publica en Subgraph Studio y se sirve desde su red descentralizada de indexadores; el plan gratuito incluye 100 000 consultas al mes. Todo lo demás del subgrafo es igual.



¿CUÁL USAR EN SU PROYECTO?

OPCIÓN	CUÁNDO CONVIENE	QUÉ CUESTA	QUÉ CENTRALIZA
queryFilter en la interfaz	Pocos eventos, un contrato, consultas simples.	Nada extra; llamadas al RPC.	Nada nuevo: el mismo RPC de siempre.
Subgrafo (The Graph)	Muchos eventos, filtros, orden, varias entidades.	Aprender GraphQL y AssemblyScript; consultas sobre el plan gratuito se pagan.	Dependen de la red de indexadores y de Studio.
Indexador propio (script + base de datos)	Necesidades muy específicas o datos fuera de la cadena.	Operar un servidor y una base de datos.	Todo: ustedes son el punto único.



REGLA DEL CURSO

Usen la pieza más simple que resuelva. Para el Avance 2 basta queryFilter en casi todos los proyectos. Si eligen un subgrafo, el README debe decir por qué queryFilter no alcanzaba: es una decisión de diseño que se defiende.

ENS: VITALIK.ETH EN VEZ DE 0XD8DA...6045

Una dirección de 42 caracteres no se puede verificar a ojo. ENS es un registro en cadena que asocia nombres a direcciones, como el DNS asocia dominios a IP. Resolverlo es una LECTURA en dos pasos.

```
RESUELTO DE VERDAD EL 17-SEP-2026, RED PRINCIPAL
```

```
const p = new ethers.JsonRpcProvider(URL_MAINNET); // la URL, desde el keystore
await p.resolveName("vitalik.eth");
// → 0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045 (nombre → dirección)
await p.lookupAddress("0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045");
// → "vitalik.eth" (dirección → nombre, «reverse»)
```

CÓMO FUNCIONA

El registro ENS dice qué contrato «resolver» atiende cada nombre; el resolver devuelve la dirección, o un contenthash para un sitio en IPFS.

ERROR TÍPICO

Creer que el nombre es para siempre: los .eth se pagan por año (5 USD/año desde 5 letras) y, si vencen, tras 90 días de gracia otro los puede registrar.



TRES FORMAS DE FIRMAR, TRES RIESGOS

En la Sesión 3 firmamos transacciones. Pero una billetera también firma MENSAJES: no van a la cadena, no cuestan gas, y aun así prueban quién los autorizó. La diferencia está en qué ve el usuario antes de firmar.

TIPO	QUÉ SE FIRMA	QUÉ VE EL USUARIO	RIESGO
Hash crudo (eth_sign)	32 bytes cualesquiera.	Un hexadecimal ilegible.	Puede ser el hash de una transacción: por eso las billeteras lo bloquean.
Texto (ERC-191, personal_sign)	Un texto con el prefijo «\x19Ethereum Signed Message».	El texto tal cual.	El prefijo impide que sea una transacción. Es el que usa SIWE.
Datos tipados (EIP-712)	Campos con tipo + un DOMINIO (app, versión, red, contrato).	Cada campo con su nombre y valor.	Si el usuario no lee los campos, puede firmar un permiso de gastar sus tokens.

La firma sigue siendo la de la Sesión 3: ECDSA sobre secp256k1, y de la firma se recupera la dirección. Lo que cambia es QUÉ bytes se firman.



DOMINIO + TIPOS + MENSAJE = UN RESUMEN

LO QUE CORRE EL LABORATORIO 13

```
const dominio = { name: "AsistenciaUSB", version: "1",  
                  chainId: 11155111, verifyingContract: "0x..." }; // Sepolia y SU contrato  
const tipos = { Asistencia: [  
  { name: "estudiante", type: "address" }, { name: "sesion", type: "uint256" },  
  { name: "nonce",      type: "uint256" }, { name: "vence",  type: "uint256" } ] };  
const firma = await signer.signTypedData(dominio, tipos, mensaje); // gratis  
ethers.verifyTypedData(dominio, tipos, mensaje, firma);           // → dirección
```

```
resumen = keccak256( 0x19 0x01 || hash(dominio) || keccak256( typeHash || estudiante || sesion || nonce || vence ) )
```

EN SOLIDITY (OPENZEPPELIN 5)

Heredan EIP712("AsistenciaUSB", "1"). _hashTypedDataV4(hashStruct) arma el resumen; ECDSA.recover(resumen, firma) devuelve quién firmó.

ERROR TÍPICO

Escribir el tipo con un espacio de más, otro orden o abi.encodePacked. La firma «no verifica» y nadie sabe por qué: el typeHash debe ser idéntico, carácter por carácter.



UNA FIRMA VÁLIDA NO ES VÁLIDA PARA SIEMPRE

Una firma es un objeto que cualquiera puede copiar. Si el mensaje no dice DÓNDE, CUÁNTAS VECES y HASTA CUÁNDO vale, quien la copie la puede reenviar. Cada campo cierra un ataque concreto, y el laboratorio prueba los cinco:

ATAQUE	QUÉ LO IMPIDE	PRUEBA DEL LABORATORIO
Reenviar la firma en otra red	chainId en el dominio.	★ dominio de otra cadena
Reenviarla a otro contrato	verifyingContract en el dominio.	★ dominio de otro contrato
Enviarla dos veces	nonce que se consume al usarla.	★ repetición (InvalidAccountNonce)
Usarla meses después	vence: fecha límite en el mensaje.	★ firma vencida
Firmar por otra persona	recover(resumen, firma) ≠ estudiante.	★ firma de otro (FirmaInvalida)

Error típico: comprobar la firma y olvidar el nonce. El contrato «funciona» en todas las pruebas felices y es vulnerable a la primera repetición.



FIRMAR UN PERMISO ES ENTREGAR LA LLAVE

EIP-712 hace legible la firma, pero no la hace inofensiva. El mismo mecanismo que hoy registra una asistencia sirve, en ERC-2612 (permit), para autorizar a un tercero a gastar tus tokens. Sin gas, sin transacción visible.

EL ATAQUE DE PHISHING

- Un sitio falso pide «firmar para verificar la billetera».
- Lo que se firma es un Permit: spender = el atacante, value = todo.
- El atacante envía la firma y vacía los tokens. La víctima nunca pagó gas.

LA COSTUMBRE PROFESIONAL

- Leer el dominio: ¿es la app que creo, en la red que creo?
- Leer el tipo: Permit, Order, Delegation... ¿qué autoriza?
- Si no se entiende, no se firma. Regla del curso desde la S1.



REGLA DEL CURSO

No se firma lo que no se entiende, aunque «sea gratis». Una firma sin gas puede mover más dinero que una transacción.



SIGN-IN WITH ETHEREUM (ERC-4361)

EL MENSAJE ESTÁNDAR (TEXTO, ERC-191)

miapp.edu.co wants you to sign **in** with your Ethereum account:
0x70997970C51812dc3A010C7d01b50e0d17dc79C8

Ingresar al tablero del curso.

URI: **https://miapp.edu.co**

Version: **1**

Chain ID: **11155111**

Nonce: **8f3kQ2mZ**

Issued At: **2026-09-17T19:30:00Z**

Expiration Time: **2026-09-17T19:40:00Z**

01

NONCE

El servidor genera un nonce de un solo uso.

02

FIRMA

La billetera muestra el texto; el usuario firma.

03

VERIFICA

El servidor recupera la dirección y revisa dominio, nonce y fecha.

04

SESIÓN

Emite una cookie o un token como siempre.



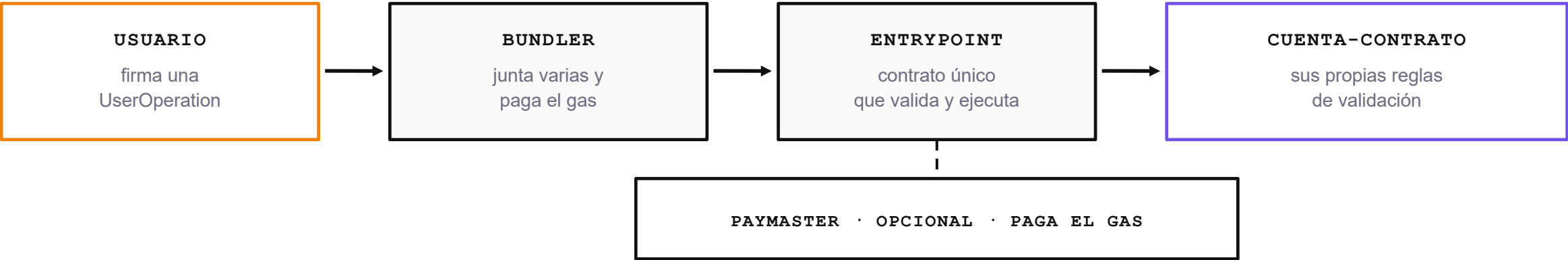
ERROR TÍPICO: «SIWE ES EIP-712»

No: ERC-4361 firma TEXTO con ERC-191, a propósito, porque todas las billeteras lo soportan. EIP-712 es otra herramienta, la que usa el laboratorio para autorizar una acción en un contrato.



ABSTRACCIÓN DE CUENTAS: ERC-4337

Una cuenta normal (EOA) es «una clave, una cuenta»: sin la frase semilla no hay recuperación, y sin ETH no hay primera transacción. ERC-4337 convierte la cuenta en un contrato programable, sin cambiar el protocolo.



CAPACIDAD	QUÉ RESUELVE PARA EL USUARIO NUEVO
Recuperación social o por guardianes	Perder un dispositivo ya no es perderlo todo.
Gas patrocinado (paymaster)	Empezar sin tener ETH: la app paga.
Operaciones por lotes	approve + usar en una sola confirmación (el doble paso de la S10).



EIP-7702: LA CUENTA NORMAL APRENDE TRUCOS

Con la actualización Pectra (red principal, 7 de mayo de 2025) una EOA puede firmar una AUTORIZACIÓN que delega su ejecución al código de un contrato. Conserva la misma dirección y los mismos fondos, y gana lotes y gas patrocinado.

	ERC-4337	EIP-7702
Qué es	Estándar de contratos, sin cambiar el protocolo.	Cambio del protocolo: un tipo de transacción nuevo.
La dirección	Una cuenta-contrato nueva, con dirección nueva.	La misma EOA de siempre.
La clave original	Puede dejar de ser necesaria (otras reglas).	Sigue mandando: quien tiene la clave, manda.
Se complementan	Aporta la infraestructura: bundlers, paymasters.	Deja que una EOA existente use esa infraestructura.



EL RIESGO NUEVO

Firmar una autorización 7702 hacia un contrato malicioso es entregarle la cuenta entera. Es la versión extrema de la lámina A.10: la billetera debe mostrar a qué código se delega, y el usuario debe entenderlo.



IPFS + ENS FRENTE A HOSTING TRADICIONAL

HOSTING TRADICIONAL

Vercel, Netlify o un servidor propio. Rápido de montar y gratis para empezar. Pero si el proveedor cae, bloquea la cuenta o cambia el código, la dApp deja de estar accesible o sirve otra cosa.

IPFS + ENS

Se compila el frontend, se sube a IPFS (contenido direccionado por hash, S11) y se pone su CID en el contenthash del nombre ENS. Un gateway como eth.limo lo sirve: vitalik.eth.limo funciona así hoy.

CAPA	¿QUIÉN LA CONTROLA EN UNA DAPP TÍPICA?
Contratos	Nadie en particular: la cadena.
Frontend	El proveedor de hosting, o nadie si está en IPFS con su CID fijado (pinning).
Gateway IPFS y RPC	Empresas concretas: siguen siendo puntos centrales.
Indexador	Studio / red de The Graph, o su propio servidor.



LA ESTRUCTURA MÍNIMA DE UN PROYECTO WEB3

ÁRBOL DE REFERENCIA

```
mi-proyecto/  
├── README.md           qué es, cómo instalar, probar, desplegar  
├── contracts/          el código fuente de los contratos  
├── test/               las pruebas (cobertura ≥ 80 %)  
├── ignition/modules/   despliegue reproducible  
├── frontend/           la interfaz (Avance 2)  
├── docs/               arquitectura, decisiones, SEGURIDAD.md  
├── package.json        y package-lock.json SIEMPRE  
└── .gitignore          node_modules, artifacts, coverage, .env
```

El README es la puerta de entrada. La plantilla está en `material/proyecto/02-avance-1`. Debe tener, como mínimo:

- Problema y por qué blockchain (del anteproyecto).
- Direcciones desplegadas con enlace a Etherscan verificado.
- Comandos exactos para instalar, probar y desplegar.
- Resultado de cobertura y de Slither.
- Qué queda centralizado, dicho con honestidad.



NINGÚN SECRETO EN EL REPOSITORIO

Claves privadas y URLs de RPC van SOLO en el keystore de Hardhat (`npx hardhat keystore set ...`). Antes de cada push: `git status` y revisar. Un secreto subido una vez queda en la historia de Git aunque después se borre.



SEIS IDEAS DE INFRAESTRUCTURA

- 01** El estado guarda el presente; la historia está en los eventos. Se lee con `queryFilter` o se indexa.

- 02** The Graph hoy se usa por Subgraph Studio: el hosted service se apagó en junio de 2024.

- 03** ENS es una lectura en dos pasos; los nombres vencen y los parecidos engañan.

- 04** EIP-712 firma datos con dominio; el nonce y el vencimiento evitan que la firma se reuse.

- 05** SIWE firma texto (ERC-191). 4337 y 7702 hacen la cuenta programable; 7702 ya está en la red.

- 06** Un repo está listo cuando otro lo corre siguiendo el README, sin secretos dentro.



PREGUNTA DE VERIFICACIÓN

Un compañero guarda en su contrato un arreglo con cada asistencia «para poder mostrar el historial», y su login pide firmar el texto «Hola». ¿Qué dos cosas le cambian?

ANTES DE RESPONDER, PIENSEN EN

- Dónde debería vivir la historia, y con qué se lee desde la interfaz.
- Qué le falta a «Hola» para que una firma copiada no sirva a un atacante.
- Qué campos del dominio EIP-712 y del mensaje SIWE cumplen esa función.

B



APROXIMADAMENTE 70 MINUTOS · 40 DE LAB + 30 DE INTEGRACIÓN

LAB 13 · FIRMA EIP-712 E INTEGRACIÓN

Primero, en parejas, un contrato que autentica con firmas y resiste cinco ataques. Después, cada equipo integra su propia dApp mientras el docente hace la revisión técnica.



ASISTENCIA FIRMADA: FIRMA UNO, PAGA OTRO

El estudiante firma en su billetera «asistí a la sesión 13», gratis. El computador del docente envía esa firma al contrato y paga el gas. El contrato solo registra la asistencia si la firma es del estudiante, para este contrato, en esta red, con el nonce que toca y antes de vencer.

ARCHIVO	QUÉ ES	QUÉ HACEN USTEDES
<code>andamiaje/s13/AsistenciaFirmada.sol</code>	El contrato con TODO 1 a 5.	Lo copian sobre contracts/s13/ y lo completan.
<code>test/s13/AsistenciaFirmada.test.js</code>	16 pruebas: 8 felices, 8 ataques (★).	No se modifica. Es el juez.
<code>scripts/s13/firmar-asistencia.js</code>	Firma, verifica fuera y envía; lee eventos.	Lo corren en local y luego en Sepolia.
<code>ignition/modules/s13-AsistenciaFirmada.js</code>	El despliegue reproducible.	Despliegan y verifican en Sepolia.

Evidencia: dirección del contrato verificada en Sepolia + hash de la transacción registrar + salida del script + respuestas de la guía.



PREPARAR Y VER LAS PRUEBAS EN ROJO

EN LA TERMINAL

```
cd laboratorios-evm
git pull                                # traer el material de hoy
copy andamiaje\s13\AsistenciaFirmada.sol contracts\s13\      (Windows)
cp andamiaje/s13/AsistenciaFirmada.sol contracts/s13/        (macOS / Linux)
npx hardhat test test/s13/AsistenciaFirmada.test.js

# 3 passing      ← las que no dependen de su código (dominio, firmar sin gas...)
# 13 failing     ← el mapa de lo que falta
```

CÓMO SABER QUE VA BIEN

- Compila sin errores (solo avisos de «pure»).
- Salen exactamente 3 verdes y 13 rojas.
- Las rojas nombran lo que falta: resumen, firma, nonce...

SI NO SALE ASÍ

- HH404 / archivo no encontrado: están fuera de laboratorios-evm.
- Error de compilación: copiaron el archivo a medias.
- 0 pruebas: pasaron una carpeta en vez del archivo.

LOS CINCO TODO DEL CONTRATO

```
LA SOLUCIÓN, PARA LA PROYECCIÓN DESPUÉS DEL PUNTO DE CONTROL

function hashAsistencia(...) public view returns (bytes32) {
    bytes32 hashStruct = keccak256(abi.encode(ASISTENCIA_TYPEHASH,
                                              estudiante, sesion, nonce, vence)); // TODO 1
    return _hashTypedDataV4(hashStruct);
}

function registrar(address estudiante, uint256 sesion, uint256 nonce,
                  uint256 vence, bytes calldata firma) external {
    if (block.timestamp > vence) revert FirmaVencida(vence, block.timestamp); // 2
    address r = ECDSA.recover(hashAsistencia(estudiante, sesion, nonce, vence), firma);
    if (r != estudiante) revert FirmaInvalida(r, estudiante); // 3
    _useCheckedNonce(estudiante, nonce); // 4
    asistio[sesion][estudiante] = true; // 5
    emit AsistenciaRegistrada(estudiante, sesion, msg.sender, nonce);
}
```

Punto de control a los 20 minutos: con el TODO 1 pasa la prueba «calcula el mismo resumen que ethers». Al terminar el 5: 16 passing.

No la copien antes de intentarlo: la guía da cada TODO con pistas, y la proyección llega después del punto de control.



EL SCRIPT: FIRMAR, VERIFICAR, ENVIAR, LEER

```
SALIDA REAL EN LA RED LOCAL (LA FIRMA CAMBIA CON LA HORA)
```

```
npx hardhat run scripts/s13/firmar-asistencia.js
```

```
3 · FIRMA (65 bytes = r || s || v)
  0x28ccf51f1807d3ec0a1170eea504e438cffeb71ae5e308ed6035...
  transacciones del estudiante antes/después de firmar: 0 / 0 ← firmar no es transaccionar
4 · VERIFICACIÓN FUERA DE LA CADENA
  recuperado      : 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 ✓ coincide
5 · ENVÍO EN CADENA
  enviada por     : 0xf39Fd6e51aad88F6F4ce6aB8827279cfffB92266 (el docente paga el gas)
  bloque / gas    : 2 / 75330
6 · EVENTOS LEÍDOS CON queryFilter
  bloque 2 · sesión 13 · nonce 0 · enviada por 0xf39F...2266
```

Lo que deben poder explicar: por qué el contador de transacciones del estudiante no cambia al firmar, y por qué la dirección que envía es otra.



LLEVARLO A SEPOLIA

```
REQUIERE EL KEYSTORE DE LA SESIÓN 8 (SEPOLIA_RPC_URL, SEPOLIA_PRIVATE_KEY, ETHERSCAN_API_KEY)

npx hardhat ignition deploy ignition/modules/s13-AsistenciaFirmada.js --network sepolia --verify
$env:CONTRATO="0x...la dirección que imprimió Ignition..." # PowerShell
npx hardhat run scripts/s13/firmar-asistencia.js --network sepolia
```

01 DESPLEGAR

Ignition imprime la dirección y, con --verify, el enlace a Etherscan con el código verificado.

02 FIRMAR Y ENVIAR

En Sepolia hay una sola cuenta: firma y envía la misma. Tarda unos 12 s por bloque.

03 COMPROBAR

En Etherscan: la pestaña Events muestra AsistenciaRegistrada con su dirección y la sesión 13.

04 ENTREGAR

Dirección, hash de la transacción registrar y captura de la pestaña Events.

LO QUE SUELE FALLAR EN ESTE LABORATORIO

SÍNTOMA	CAUSA Y SOLUCIÓN
El resumen no coincide con el de ethers	typeHash distinto: espacios, orden o nombres. Cópienlo exacto de la guía.
FirmaInvalida con una firma correcta	Firmaron con otro chainId o verifyingContract: lean el dominio con eip712Domain().
InvalidAccountNonce en la primera firma	Usaron un nonce fijo: pídanlo con contrato.nonces(estudiante).
FirmaVencida en Sepolia	La hora del bloque, no la del computador: calculen vence desde el último bloque.
queryFilter devuelve vacío en Sepolia	Rango desde el bloque 0: el RPC lo rechaza o lo corta. Empiecen cerca del despliegue.

Todos estos síntomas son la misma lección: una firma vale solo para el mensaje EXACTO que se firmó. Un bit distinto y es otra firma.

INTEGRACIÓN ASISTIDA: EL CORTE VERTICAL

- | | | |
|-----------|--------------------------|---|
| 01 | CONTRATOS | Desplegados y verificados en Sepolia: esto es Avance 1 y se revisa hoy. |
| 02 | PRUEBAS | <code>npx hardhat test --coverage</code> con 80 % o más de líneas en SUS contratos. |
| 03 | LECTURA | La interfaz muestra un dato real de su contrato (avance hacia el Avance 2). |
| 04 | ESCRITURA | Una transacción desde la interfaz, con sus estados (S12). |
| 05 | FIRMA (SI APLICA) | ¿Algún flujo de su proyecto mejora con EIP-712? Anótenlo en el README. |

Mientras tanto, el docente pasa equipo por equipo con la lista pública de preguntas de la revisión técnica (bloque C). Tengan abiertos Etherscan, la terminal y el README.



CASI TODO ES UN DETALLE OLVIDADO

SÍNTOMA	CAUSA FRECUENTE	SESIÓN DONDE SE VIO
La interfaz no ve el contrato	Dirección o ABI viejos, o red distinta a la del despliegue.	S12
Cantidades enormes o minúsculas	Olvidar parseUnits / formatUnits con los decimales del token.	S10
La transacción «no hace nada»	No se espera tx.wait() y se lee el estado antes de que se mine.	S12
La imagen del NFT no aparece	tokenURI con un CID mal fijado o sin gateway.	S11
Funcionaba y de repente no	El proveedor RPC cortó por límite de tasa: clave propia en el keystore.	S12

Por eso este bloque no es expositivo: el docente circula y resuelve los atascos reales, que casi nunca son conceptos nuevos.



PREGUNTA DE VERIFICACIÓN

Ana firmó su asistencia en Sepolia para SU contrato. Beto copia la firma y la envía al contrato de su equipo, desplegado con el mismo código. ¿Pasa? ¿Y si la envía dos veces al contrato de Ana?

ANTES DE RESPONDER, PIENSEN EN

- Qué campo del dominio distingue dos contratos con el mismo código.
- Qué prueba con ★ del laboratorio reproduce cada caso.
- Qué error exacto devuelve el contrato en cada uno.

C



APROXIMADAMENTE 25 MINUTOS · 10 % DE LA NOTA FINAL

ENTREGA DEL AVANCE 1

La primera entrega grande del proyecto: contratos desplegados y verificados, pruebas con cobertura y un repositorio que otro puede correr.



LOS TRES COMPONENTES DEL AVANCE 1

COMPONENTE	REQUISITO VERIFICABLE	CÓMO SE COMPRUEBA
Contratos en Sepolia	Desplegados con Ignition y verificados: el código se lee en Etherscan.	Enlace en el README; pestaña Contract con marca verde.
Suite de pruebas	≥ 80 % de líneas en SUS contratos, con casos de reversión y límite.	npx hardhat test --coverage en vivo.
Repositorio documentado	README con la plantilla, sin secretos, reproducible desde cero.	El docente clona y sigue el README.

QUÉ NO ES EL AVANCE 1

- El frontend terminado: eso es el Avance 2 (S16).
- Cobertura de librerías de OpenZeppelin: cuentan sus contratos.

CÓMO SE ENTREGA

- Enlace al repositorio y commit etiquetado avance-1.
- Formulario del curso, hoy antes de las 23:59.



RÚBRICA DEL AVANCE 1

CRITERIO	PESO	EXCELENTE (5,0)	INSUFICIENTE (< 3,0)
Contratos desplegados y verificados	30 %	Todos verificados, desplegados con Ignition, direcciones en el README.	Sin desplegar o sin verificar.
Pruebas y cobertura	35 %	≥ 80 % de líneas; reversiones, límites y eventos probados.	Menos de 60 % o solo casos felices.
Repositorio y README	25 %	Se instala y prueba siguiendo el README; sin secretos; Slither documentado.	No arranca o contiene claves.
Revisión técnica en vivo	10 %	Los tres integrantes responden con solvencia.	Nadie sabe explicar su código.

La columna «aceptable (3,5)» y el detalle de cada nivel están en el documento del Avance 1. Una clave privada en el repositorio anula el criterio de repositorio.



LAS PREGUNTAS QUE HARÁ EL DOCENTE

1. Muéstrenme el contrato verificado en Etherscan. ¿Qué hace esta función, línea por línea?
1. Corran la cobertura ahora. ¿Qué línea NO está cubierta y por qué aceptan ese riesgo?
1. Enséñenme una prueba que espere una reversión. ¿Qué error personalizado verifica?
1. ¿Qué dijo Slither y qué hicieron con cada hallazgo?
1. Si clono el repo ahora, ¿qué comando corro primero? ¿Dónde están las claves?
1. ¿Qué parte de su proyecto NO está descentralizada, y lo dice el README?

No es un examen sorpresa: la lista completa está publicada. Prepárenla. Cualquiera de los tres puede ser preguntado.



LA LISTA DE CHEQUEO EN CINCO MINUTOS

CÓDIGO Y PRUEBAS

- npx hardhat test pasa completo en un clon limpio.
- Cobertura ≥ 80 % de líneas en sus contratos.
- Al menos una prueba por cada error personalizado.
- Slither corrido; hallazgos anotados en docs/SEGURIDAD.md.

DESPLIEGUE Y REPOSITORIO

- Direcciones de Sepolia en el README, con enlace verificado.
- package-lock.json incluido; .gitignore correcto.
- git log sin claves ni .env; keystore para todo secreto.
- Etiqueta avance-1 empujada al remoto.



LA PRUEBA DEL README

Uno de los tres clona el repositorio en una carpeta nueva y sigue el README sin ayuda de los otros dos. Si se detiene en algún paso, el README no está listo. Háganlo antes de entregar, no después.



LA PREGUNTA DE LA SEMANA

¿Qué pieza de hoy —indexación, ENS, EIP-712, SIWE o abstracción de cuentas— le haría más bien a su proyecto, y qué costo o centralización agrega?

PARA LA PRÓXIMA SESIÓN

- Entregar el Avance 1 (contratos verificados, cobertura $\geq 80\%$, README) con la etiqueta avance-1.
- Laboratorio 13: evidencia de Sepolia y respuestas de la guía, por el canal del curso.
- Leer para la Sesión 14: qué es un AMM ($x \cdot y = k$) y qué fue el colapso de UST/Terra.
- Seguir el frontend: el Avance 2 (dApp integrada) se entrega en la Sesión 16.



LO QUE SE LLEVAN DE HOY

Una demo necesita un contrato y una pantalla. Un producto necesita todo lo que hay en medio.

Con el Avance 1 cierra la Unidad III. La Unidad IV mira el ecosistema: DeFi, gobernanza, escalabilidad y la ley.

SESIÓN 14 · DEFI · AMM, PRÉSTAMOS, STABLECOINS, ORÁCULOS Y SUS RIESGOS